

Guideline

This document serves as a guideline for reproducing the numerical results via Terminal.

1. Download the Files from Github

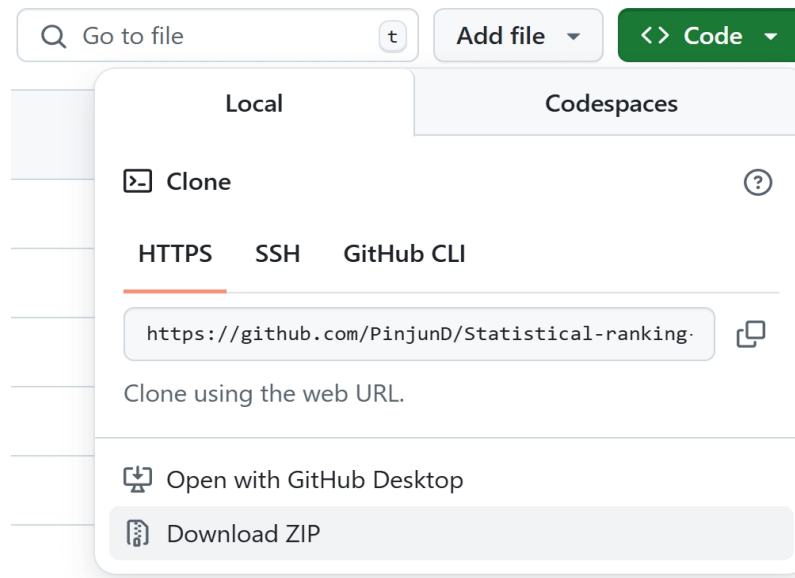
If you have already installed *Git Bash*, you can use it to clone the repository in Terminal:

```
git clone https://github.com/PinjunD/Statistical-ranking-with-dynamic-covariate.git
cd Statistical-ranking-with-dynamic-covariate
```

```
D:\>git clone https://github.com/PinjunD/Statistical-ranking-with-dynamic-covariate.git
Cloning into 'Statistical-ranking-with-dynamic-covariate'...
remote: Enumerating objects: 1124, done.
remote: Counting objects: 100% (382/382), done.
remote: Compressing objects: 100% (247/247), done.
remote: Total 1124 (delta 215), reused 266 (delta 130), pack-reused 742 (from 1)
Receiving objects: 100% (1124/1124), 129.41 MiB | 3.44 MiB/s, done.
Resolving deltas: 100% (660/660), done.

D:\>cd Statistical-ranking-with-dynamic-covariate
D:\Statistical-ranking-with-dynamic-covariate>
```

Otherwise, please click the button *Download ZIP*:



2. Confirm the Suitability of the Environment

Please access the Python environment and verify the package versions (in Terminal).

```
python
import numpy
import pandas
```

```
import matplotlib
import joblib

print(numpy.__version__) # Required: at least 1.26.4
print(pandas.__version__) # Required: at least 2.2.1
print(matplotlib.__version__) # Required: at least 3.9.0
print(joblib.__version__) # Required: at least 1.4.2
```

```
D:\Statistical-ranking-with-dynamic-covariate>python
Python 3.9.9 (tags/v3.9.9:ccb0e6a, Nov 15 2021, 18:08:50) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
>>> import pandas
>>> import matplotlib
>>> import joblib
>>>
>>> print(numpy.__version__) # Required: at least 1.26.4
1.26.4
>>> print(pandas.__version__) # Required: at least 2.2.1
2.2.1
>>> print(matplotlib.__version__) # Required: at least 3.9.0
3.9.0
>>> print(joblib.__version__) # Required: at least 1.4.2
1.4.2
>>> quit()
```

If you have not installed the packages, you might get the following error messages:

```
D:\Statistical-ranking-with-dynamic-covariate>python
Python 3.9.9 (tags/v3.9.9:ccb0e6a, Nov 15 2021, 18:08:50) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'numpy'
>>> import pandas
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'pandas'
>>> import matplotlib
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'matplotlib'
>>> import joblib
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'joblib'
```

You can install the required packages using the following code:

```
pip install numpy pandas matplotlib joblib
```

If your packages do not meet the required version, please upgrade them as follow:

```
pip install --upgrade pandas numpy joblib matplotlib
```

3. Tennis Data

```
cd Tennis
```

- First, please access the directory ``../Tennis``:

```
D:\Statistical-ranking-with-dynamic-covariate>cd Tennis
D:\Statistical-ranking-with-dynamic-covariate\Tennis>
```

- To reproduce Figure 1 and Figure 5, please execute the code below:

```
python "Figure1.py" # In Tennis
python "Figure5.py" # In Tennis
```

- The following output will be displayed (for Figure 1):

```
D:\Statistical-ranking-with-dynamic-covariate\Tennis>python Figure1.py
In this program, we plot the log-scores of tennis players over time.

=====Data Loading=====
Loading data from 'data\tennis(preprocessed).csv'...
Complete!

=====Data analysis=====
We have selected the optimal basis functions based on the BIC.
Here,  $(a,p) \in \{(25,0.01), (25,0.03), (30,0.01), (35,0.01)\}$ .
We use PlusDC to fit the data using various combinations of the four bases.
Running...
Complete!  $v = [ 6.23852629 -0.86541806 -3.33848892 3.31885836]$ 
Next, we fit the PlusDC model and identify the top 10 players.
=====
top-1: player: Roger Federer, score: 3.673056180031579
top-2: player: Novak Djokovic, score: 3.638065846521144
top-3: player: Rafael Nadal, score: 3.6009231949782974
top-4: player: Jimmy Connors, score: 3.00658446074499
top-5: player: Andre Agassi, score: 2.9915879425760696
top-6: player: Ivan Lendl, score: 2.962388824938805
top-7: player: Carlos Alcaraz, score: 2.9306540982638944
top-8: player: Pete Sampras, score: 2.929052899380519
top-9: player: Andy Murray, score: 2.8610323175888537
top-10: player: John McEnroe, score: 2.8175553253327816
Loading birthday information...
Complete!
Plot...
Complete!
We have saved the results as Figure1.pdf.
```

* The final results will be saved to the ".../results" folder by default.

4. Horse-Racing Data

```
cd ..  
cd HorseRacing
```

- Please access the directory "../HorseRacing":

```
D:\Statistical-ranking-with-dynamic-covariate\Tennis>cd..  
  
D:\Statistical-ranking-with-dynamic-covariate>cd HorseRacing
```

- To reproduce TableS2, Figure S2 and Figure S3, please execute the code below:

```
python "TableS2.py" # In HorseRacing  
python "FigureS2.py" # In HorseRacing  
python "FigureS3.py" # In HorseRacing
```

- You will observe the following output:

```
D:\Statistical-ranking-with-dynamic-covariate\HorseRacing>python TableS2.py  
In this program, we investigate the impact of covariates on horse racing results.  
Specifically, we consider three factors to explain the internal scores of horses:  
(1)Actual weight: The weight that a horse carries during a competition.  
(2)Draw: The post position number assigned to a horse in a race.  
(3)Public belief: The winning probability derived from the final win odds shortly before the competition begins.  
  
=====Data Loading=====  
To begin with, we load the data from 'data\runs(preprocessed).csv'...  
|-----| 100.0% Complete
```

* Similarly, the final results will be saved to the "../results" folder by default.

5. Simulation

- Please access the directory "../Simulation":

```
cd ..  
cd Simulation
```

```
D:\Statistical-ranking-with-dynamic-covariate\HorseRacing>cd ..  
  
D:\Statistical-ranking-with-dynamic-covariate>cd Simulation
```

- To reproduce Figure S1, please execute the following code:

```
python "FigureS1.py" # In Simulation
```

- Below is the expected output:

```
D:\Statistical-ranking-with-dynamic-covariate\Simulation>python FigureS1.py
In this program, we verify the uniform consistency of the MLE in the PlusDC model using simulated data.
We set n={200,400,600,800,1000} and conduct the simulations for NURHM and HSBM with 300 repetitions.
Please note that the program may take a considerable amount of time to complete.
Start!
=====NURHM=====
-----
The 1st repetition of NURHM: number of nodes:200.
u_infty:0.48665379266740716, v_infty:0.0066815124805834885.
-----
The 2nd repetition of NURHM: number of nodes:200.
u_infty:0.5115432850077104, v_infty:0.01771210625340358.
-----
The 3rd repetition of NURHM: number of nodes:200.
u_infty:0.6045637595370839, v_infty:0.011324221751249221.
-----
The 4th repetition of NURHM: number of nodes:200.
u_infty:0.6024801875842916, v_infty:0.03763539573611174.
```

* Similarly, the final results will be saved to the ".../results" folder by default. * Please note that the simulation may take a considerable amount of time to complete, as it involves 5 different settings in the NURHM and HSBM models, each with 300 repetitions.